PANDUAN IMPLEMENTASI · v1.0

Panduan Setup Server Lawu-2.5-Fine

Panduan komprehensif 20 langkah dari server kosong sampai production-ready untuk Debian 13 dengan Rust + Axum + Tantivy + llama.cpp (Pure CPU).

Debian 13 · Linux 6.x | Rust · Axum · Tantivy · Rayon | llama.cpp GGUF (Q4_K_M) | Pure CPU · No GPU | Open Source (MIT/Apache)

Target Hardware

Intel Core i5-10400 (6 Core / 12 Thread · AVX2 · 2.9–4.3 GHz) · **16 GB DDR4** · **SSD 128 GB** (OS — Debian 13) · **HDD 500 GB** (DATA — koleksi ebook & index Tantivy).

Lawu-2.5-Fine

AI Model Cerdas, Irit Budget · Muslim Ahlussunnah wal Jama'ah

Versi Dokumen
1.0 · 2025

Daftar Isi

20 langkah komprehensif — dari pembersihan server hingga go-live checklist.

0 Bersihkan Server Sebelum Instalasi

1 Prerequisites & Update Sistem

2 Setup User lawu & Struktur Direktori

3 Mount & Format HDD/SSD

4 Firewall (UFW) & SSH Hardening

5 Ringkasan Arsitektur

6 Spesifikasi Infrastruktur & Kernel Tuning (sysctl)

7 Install Rust Toolchain

8 Proyek Cargo — lawu-server

9 Skema Full-Text Tantivy

10 Endpoint API Backend (Axum)

11 Build llama.cpp (Pure CPU)

12 Konversi & Kuantisasi Model GGUF

13 Serve llama.cpp (OpenAI-compatible)

14 Unit systemd (Auto-restart & Boot)

15 Otomasi Pemantauan & Operasional

16 Pengujian Beban (Load Testing k6)

17 Reverse Proxy Nginx

18 Integrasi dengan Dashboard

19 Final Go-Live Checklist

Urutan Eksekusi yang Disarankan

Langkah 0 (bersihkan) → 1 (prerequisites) → 2 (user & dir) → 3 (disk) → 4 (firewall) → 5-6 (arsitektur & sysctl) → 7 (Rust) → 8-10 (Axum API) → 11-13 (llama.cpp) → 14 (systemd) → 15 (otomasi) → 16 (load test) → 17 (Nginx) → 18 (integrasi) → 19 (go-live checklist).

0 Bersihkan Server Sebelum Instalasi

Jika server pernah dipakai project Lawu versi lama, jalankan langkah ini DULU. Skip jika server masih fresh.

> ⚠ **Peringatan Data-Loss — Baca sebelum eksekusi!**
>
> Langkah **0.D** & **0.E** menghapus permanen direktori, binary, model GGUF, dan user `lawu`. Pastikan Anda menjalankan **0.C (backup)** terlebih dahulu jika ada data yang ingin disimpan.

0.A-audit-server.sh — SH

```
# 0.A — Audit server kotor (JALANKAN DULU, lihat apa yang ada sebelum bersihkan)
# Paten 3: murni CPU, tanpa GPU — audit hanya mencari sisa project Lawu lama

echo "=== AUDIT SERVER LAWU LAMA ==="

echo "1. Proses yang dengarkan port Lawu (80/8080/8081):"
sudo ss -tlnp | grep -E ':(80|8080|8081)\b' || echo "   (kosong)"

echo "2. Service systemd Lawu/llama lama:"
systemctl list-units --type=service --all | grep -iE 'lawu|llama|tantivy' || echo "   (kosong)"
systemctl list-unit-files | grep -iE 'lawu|llama|tantivy' || echo "   (tidak ada unit file)"

echo "3. Cron jobs Lawu lama:"
crontab -l 2>/dev/null | grep -iE 'lawu|llama' || echo "   (user crontab kosong)"
sudo crontab -l -u lawu 2>/dev/null || echo "   (crontab user lawu tidak ada)"
ls /etc/cron.d/ 2>/dev/null | grep -i lawu || echo "   (/etc/cron.d bersih)"

echo "4. User lawu lama:"
id lawu 2>/dev/null || echo "   (user lawu tidak ada)"
grep -E '^lawu:' /etc/passwd || echo "   (tidak ada di /etc/passwd)"

echo "5. Direktori & binary Lawu lama:"
ls -la /opt/lawu 2>/dev/null || echo "   /opt/lawu tidak ada"
ls -la /mnt/data/ebooks 2>/dev/null || echo "   /mnt/data/ebooks tidak ada"
ls -la /mnt/data/tantivy_index 2>/dev/null || echo "   /mnt/data/tantivy_index tidak ada"
ls -la ~/llama.cpp 2>/dev/null || echo "   ~/llama.cpp tidak ada"
ls -la ~/.cargo/bin/lawu-server 2>/dev/null || echo "   ~/.cargo/bin/lawu-server tidak ada"
ls -la ~/models/lawu-2.5-fine-*.gguf 2>/dev/null || echo "   ~/models tidak ada model GGUF Lawu"

echo "6. sysctl & Nginx config Lawu lama:"
ls /etc/sysctl.d/ 2>/dev/null | grep -i lawu || echo "   (sysctl bersih)"
ls /etc/nginx/sites-enabled/ 2>/dev/null | grep -i lawu || echo "   (nginx bersih)"

echo "7. Log Lawu lama:"
ls /var/log/lawu 2>/dev/null || echo "   /var/log/lawu tidak ada"
```

0.B-stop-services.sh — SH

```
# 0.B — Stop & disable service Lawu/llama lama
# HATI-HATI: ini menghentikan service Lawu lama yang sedang berjalan

# 1) Stop & disable semua service systemd Lawu/llama lama
for svc in llama-lawu lawu-2.5-fine lawu tantivy; do
  sudo systemctl stop "$svc" 2>/dev/null || true
  sudo systemctl disable "$svc" 2>/dev/null || true
done

# 2) Hapus unit file systemd lama (daemon-reload setelahnya)
sudo rm -f /etc/systemd/system/llama-lawu.service
sudo rm -f /etc/systemd/system/lawu-2.5-fine.service
sudo rm -f /etc/systemd/system/lawu.service
sudo rm -f /etc/systemd/system/tantivy.service
sudo systemctl daemon-reload

# 3) Kill proses yang masih dengarkan port Lawu (8080 inference, 8081 API)
sudo ss -tlnp | grep -E ':(8080|8081)\b' \
  | grep -oP 'pid=\K[0-9]+' | sort -u \
  | while read -r pid; do sudo kill "$pid" 2>/dev/null || true; done

# 4) Hapus cron jobs Lawu lama (user crontab + /etc/cron.d)
sudo crontab -r -u lawu 2>/dev/null || true
crontab -l 2>/dev/null | grep -v -E '/opt/lawu/|lawu-2.5-fine' | crontab - 2>/dev/null || true
sudo rm -f /etc/cron.d/lawu* 2>/dev/null
```

0.C-backup-old.sh SH

```sh
# 0.C — Backup data Lawu lama (OPSIONAL, sebelum hapus permanen)
# Jalankan ini JIKA Anda ingin jaga-jaga ada data ebook/index/markdown lama
# yang belum dimigrasi. Jika server benar-benar kosong/scrapped, lewati langkah ini.

STAMP=$(date +%Y%m%d_%H%M)
BACKUP_DIR="/mnt/data/backup-pre-clean-$STAMP"
sudo mkdir -p "$BACKUP_DIR"

# Backup koleksi ebook, index Tantivy, dan markdown hasil ekstraksi
sudo tar -czf "$BACKUP_DIR/lawu-old-data.tar.gz" \
  /opt/lawu \
  /mnt/data/ebooks \
  /mnt/data/tantivy_index \
  /mnt/data/processed_markdowns \
  2>/dev/null || true

# Backup config systemd & sysctl lama (arsip referensi)
sudo tar -czf "$BACKUP_DIR/lawu-old-configs.tar.gz" \
  /etc/systemd/system/llama-lawu.service \
  /etc/systemd/system/lawu-2.5-fine.service \
  /etc/sysctl.d/99-lawu-fine.conf \
  /etc/nginx/sites-available/lawu \
  2>/dev/null || true

sudo du -sh "$BACKUP_DIR"/* 2>/dev/null
sudo chmod -R lawu:lawu "$BACKUP_DIR" 2>/dev/null || true
echo "Backup Lawu lama tersimpan di: $BACKUP_DIR"
```

0.D-remove-files.sh SH

```sh
# 0.D — Hapus direktori & binary Lawu lama
# PERINGATAN: langkah ini MENGHAPUS PERMANEN. Pastikan 0.C (backup) sudah dilakukan.

# 1) Hapus direktori aplikasi Lawu (/opt/lawu berisi skrip watch/backup/reindex)
sudo rm -rf /opt/lawu

# 2) Hapus binary lawu-server lama
rm -f ~/.cargo/bin/lawu-server

# 3) Hapus source tree llama.cpp lama (akan di-clone ulang di Langkah 11)
rm -rf ~/llama.cpp

# 4) Hapus model GGUF Lawu lama (HATI-HATI — file besar)
#    Jika ingin simpan model agar tidak perlu re-download/re-quant, COMMENT baris ini.
rm -f ~/models/lawu-2.5-fine-*.gguf

# 5) Hapus index Tantivy & markdown hasil ekstraksi lama
#    (JIKA ingin mulai indexing dari nol. Jika ingin pakai index lama, COMMENT baris ini.)
sudo rm -rf /mnt/data/tantivy_index
sudo rm -rf /mnt/data/processed_markdowns

# 6) Kosongkan incoming/ (file ebook yang belum diproses)
# sudo rm -rf /mnt/data/ebooks/incoming/*
```

0.E-remove-user.sh SH

```sh
# 0.E — Hapus user & group 'lawu' lama
# User ini akan dibuat ulang bersih di Langkah 2.
# Jika server Anda TIDAK pernah pakai user 'lawu' sebelumnya, lewati langkah ini.

# 1) Pastikan tidak ada proses yang masih berjalan sebagai user lawu
sudo pkill -u lawu 2>/dev/null || true
sleep 2

# 2) Hapus user lawu lama BESERTA home directory-nya (-r)
sudo userdel -r lawu 2>/dev/null || true

# 3) Hapus group 'lawu' jika masih tersisa
sudo groupdel lawu 2>/dev/null || true

# 4) Verifikasi tidak ada file sisa milik UID lawu lama (orphaned files)
sudo find / -path /proc -prune -o -path /sys -prune -o -path /dev -prune \
  -path /run -prune -o -nouser -print 2>/dev/null | head -20 || true
```

0.F-clean-configs.sh SH

```sh
# 0.F — Bersihkan sysctl, Nginx config, dan log Lawu lama

# 1) Hapus sysctl custom Lawu lama (akan dibuat ulang bersih di Langkah 6)
sudo rm -f /etc/sysctl.d/99-lawu-fine.conf
sudo sysctl --system 2>/dev/null || true

# 2) Hapus Nginx config Lawu lama (akan dibuat ulang bersih di Langkah 17)
sudo rm -f /etc/nginx/sites-enabled/lawu
sudo rm -f /etc/nginx/sites-available/lawu
sudo nginx -t 2>/dev/null && sudo systemctl reload nginx 2>/dev/null || true

# 3) Bersihkan log Lawu lama (file log + entry journald lama)
sudo rm -rf /var/log/lawu
sudo journalctl --rotate 2>/dev/null || true
sudo journalctl --vacuum-time=3d 2>/dev/null || true

# 4) (OPSIONAL) Bersihkan cache build Cargo lama — hemat SSD 128GB
rm -rf ~/.cargo/registry/cache 2>/dev/null || true
rm -rf ~/.cargo/registry/src 2>/dev/null || true
rm -rf target 2>/dev/null || true
```

0.G-verify-clean.sh SH

```sh
# 0.G — Verifikasi server bersih (FINAL CHECK)
# Jalankan setelah 0.A–0.F selesai. Semua check harus cetak "✓ Bersih".

echo "=== AUDIT FINAL — SERVER BERSIH? ==="

echo "1. Port Lawu (8080/8081 harus kosong):"
sudo ss -tlnp | grep -E ':(8080|8081)\b' && echo "   ✗ MASIH ADA proses!" || echo "   ✓ Bersih"

echo "2. Service systemd Lawu/llama (harus kosong):"
systemctl list-units --type=service --all | grep -iE 'lawu|llama' && echo "   ✗ MASIH ADA service!" || echo "   ✓ Bersih"
systemctl list-unit-files | grep -iE 'lawu|llama' && echo "   ✗ MASIH ADA unit file!" || echo "   ✓ Bersih"

echo "3. User lawu (harus tidak ada):"
id lawu 2>/dev/null && echo "   ✗ MASIH ADA user!" || echo "   ✓ Bersih"

echo "4. Direktori Lawu (harus tidak ada):"
ls /opt/lawu ~/llama.cpp ~/.cargo/bin/lawu-server 2>/dev/null && echo "   ✗ MASIH ADA file!" || echo "   ✓ Bersih"

echo "5. Cron Lawu (harus kosong):"
crontab -l 2>/dev/null | grep -iE 'lawu|llama' && echo "   ✗ MASIH ADA cron!" || echo "   ✓ Bersih"
sudo crontab -l -u lawu 2>/dev/null && echo "   ✗ MASIH ADA cron lawu!" || echo "   ✓ Bersih"
ls /etc/cron.d/ | grep -i lawu && echo "   ✗ MASIH ADA cron.d!" || echo "   ✓ Bersih"

echo "6. sysctl Lawu (harus tidak ada):"
ls /etc/sysctl.d/ | grep -i lawu && echo "   ✗ MASIH ADA sysctl!" || echo "   ✓ Bersih"

echo "7. Nginx Lawu (harus tidak ada):"
ls /etc/nginx/sites-enabled/lawu 2>/dev/null && echo "   ✗ MASIH ADA nginx config!" || echo "   ✓ Bersih"

echo "8. Log Lawu (harus tidak ada):"
ls /var/log/lawu 2>/dev/null && echo "   ✗ MASIH ADA log!" || echo "   ✓ Bersih"

echo ""
echo "=========================================="
echo "✅ SERVER BERSIH — siap instalasi Lawu-2.5-Fine."
echo "   Lanjut ke Langkah 1: Prerequisites & Update Sistem."
echo "=========================================="
```

1 Prerequisites & Update Sistem

Update Debian 13 ke versi terbaru, install build dependencies (gcc, cmake, nginx, ufw, fail2ban, python venv untuk MarkItDown). Semua paket open source dari Debian main repo.

1.A-apt-update.sh SH

```sh
# 1.A — Update & upgrade sistem Debian 13 (trixie)
# Jalankan sebagai user biasa dengan sudo. Pastikan internet stabil.

sudo apt update && sudo apt full-upgrade -y
sudo apt autoremove -y

# Verifikasi versi Debian
cat /etc/debian_version
# Harus cetak: 13.x (trixie)
```

1.B-apt-install-deps.sh SH

```sh
# 1.B — Install build dependencies & tools
# Semua paket ini open-source (Debian main repo) — Paten 5: open source terpenuhi.

sudo apt install -y \
  build-essential \
  cmake \
  ninja-build \
  pkg-config \
  git \
  curl \
  wget \
  unzip \
  tar \
  ca-certificates \
  libssl-dev \
  libcurl4-openssl-dev \
  libsqlite3-dev \
  procps \
  nginx \
  ufw \
  fail2ban \
  inotify-tools \
  python3 \
  python3-pip \
  python3-venv \
  jq \
  htop \
  btop \
  ncdu \
  strace \
  lsof \
  sqlite3

# Verifikasi paket kunci terpasang
cmake --version | head -1
gcc --version | head -1
nginx -v 2>&1
ufw version
```

1.C-python-venv-markitdown.sh SH

```sh
# 1.C — Python virtualenv untuk MarkItDown (ekstraksi teks ebook)
# Paten 5: MarkItDown = MIT (Microsoft) — open source, aman dipakai.
# JANGAN pakai pip global (PEP 668 protected). Gunakan venv.

python3 -m venv ~/venvs/lawu
source ~/venvs/lawu/bin/activate

# Install MarkItDown di dalam venv (tidak mengganggu system Python)
pip install --upgrade pip
pip install markitdown

# Tes MarkItDown — harus cetak teks dari PDF/DOCX dummy
echo "# test" > /tmp/test.md
markitdown /tmp/test.md | head -5
deactivate
```

2 Setup User lawu & Struktur Direktori

Buat system user 'lawu' (isolasi privilege), struktur direktori aplikasi di SSD (/opt/lawu) & data di HDD (/mnt/data).

2.A-create-user-lawu.sh SH

```
# 2.A — Buat user & group 'lawu' (system user, no sudo)
# >>> RUN AS: user admin Anda (yang punya sudo) <<<
# User 'lawu' akan menjalankan semua service Lawu — isolasi privilege (Paten keamanan).
# User lawu TIDAK diberi sudo (intentional — principle of least privilege).

# 1) Buat group 'lawu' jika belum ada
sudo groupadd -f lawu

# 2) Buat user system 'lawu' dengan home directory
sudo useradd -m -d /home/lawu -s /bin/bash -g lawu lawu

# 3) Set password kuat untuk user lawu (untuk 'su - lawu' saat debugging dari user admin)
sudo passwd lawu

# 4) Verifikasi user dibuat (tanpa sudo di grup)
id lawu
ls -ld /home/lawu

# 5) VERIFIKASI lawu TIDAK di sudoers (ini BENAR — by design)
sudo -u lawu sudo -n whoami 2>&1 | grep -q 'not in the sudoers' \
  && echo "✓ lawu TIDAK punya sudo (correct — service user)" \
  || echo "✗ lawu punya sudo (HAPUS: sudo deluser lawu sudo)"

# 6) Untuk setup service nanti, gunakan 'sudo -u lawu <cmd>' dari user admin
#    Contoh: sudo -u lawu git clone ... (build sebagai lawu, tanpa sudo)
# Atau switch ke lawu: 'sudo su - lawu' lalu exit kembali ke admin

# CATATAN: Semua perintah sudo di Langkah 0-6, 14-17 jalankan sebagai USER ADMIN.
#          Langkah 7-13 (Rust build, llama.cpp build) jalankan sebagai USER LAWU
#          (sudo su - lawu, tanpa sudo prefix).
```

2.B-create-directories.sh SH

```
# 2.B — Buat struktur direktori Lawu & set ownership
# >>> RUN AS: user admin (yang punya sudo) <<<

# Direktori APLIKASI di SSD 128GB (OS) — cepat, untuk binary & config
sudo mkdir -p /opt/lawu
sudo mkdir -p /opt/lawu/bin
sudo mkdir -p /opt/lawu/scripts
sudo mkdir -p /opt/lawu/logs

# Direktori DATA di HDD 500GB — koleksi ebook, index, markdown, backup, MODELS
sudo mkdir -p /mnt/data/ebooks
sudo mkdir -p /mnt/data/ebooks/incoming
sudo mkdir -p /mnt/data/ebooks/processed
sudo mkdir -p /mnt/data/processed_markdowns
sudo mkdir -p /mnt/data/tantivy_index
sudo mkdir -p /mnt/data/backup
sudo mkdir -p /mnt/data/models      # ← SEMUA model GGUF disimpan di HDD (SSD hanya utk OS)

# Direktori di home user lawu (source tree llama.cpp di SSD — cepat utk build)
sudo -u lawu mkdir -p /home/lawu/src

# SYMLINK: ~/models → /mnt/data/models
# Tujuan: model file fisik di HDD (hemat SSD), tapi path ~/models tetap bisa dipakai
sudo -u lawu rm -rf /home/lawu/models
sudo -u lawu ln -s /mnt/data/models /home/lawu/models
sudo -u lawu ls -la /home/lawu/models  # harus cetak: symlink → /mnt/data/models

# Set ownership: semua direktori Lawu milik user lawu
sudo chown -R lawu:lawu /opt/lawu
sudo chown -R lawu:lawu /mnt/data

# Set permission: 755 untuk dir, 640 untuk file (principle least privilege)
sudo find /opt/lawu -type d -exec chmod 755 {} \;
sudo find /opt/lawu -type f -exec chmod 640 {} \;
sudo find /mnt/data -type d -exec chmod 755 {} \;

# Verifikasi
ls -ld /opt/lawu /mnt/data/ebooks /mnt/data/tantivy_index /mnt/data/backup
ls -ld /home/lawu /home/lawu/src /home/lawu/models
```

3 Mount & Format HDD/SSD

Identifikasi SSD (OS) vs HDD (DATA), format HDD 500GB sebagai ext4, mount otomatis via fstab, tuning SSD (TRIM, I/O scheduler).

> **Hati-hati memilih disk!**
>
> Pastikan `/dev/sdb` di Langkah 3.B BENAR-BENAR HDD data kosong, bukan disk OS. Salah format = kehilangan sistem. Selalu verifikasi dengan **3.A** dulu sebelum 3.B.

3.A-identify-disks.sh — SH

```
# 3.A — Identifikasi disk SSD (OS) & HDD (DATA)
# SSD 128GB biasanya sudah jadi disk OS (sda). HDD 500GB untuk DATA.

# 1) List semua block device
lsblk -o NAME,SIZE,TYPE,MOUNTPOINT,FSTYPE,MODEL

# 2) Cek disk terpasang
sudo fdisk -l | grep -E '^Disk /dev/(sd|nvme)'

# 3) Identifikasi SSD vs HDD berdasarkan rotasi (0=SSD, 1=HDD)
for dev in /sys/block/sd* /sys/block/nvme*; do
  [ -d "$dev" ] || continue
  name=$(basename "$dev")
  rotational=$(cat "$dev/queue/rotational" 2>/dev/null)
  size=$(lsblk -dn -o SIZE "/dev/$name" 2>/dev/null)
  [ "$rotational" = "0" ] && echo "$name (SSD, $size)" || echo "$name (HDD, $size)"
done

# CONTOH OUTPUT:
# sda (SSD, 128G)   ← disk OS, jangan diformat!
# sdb (HDD, 500G)   ← disk DATA, akan di-mount ke /mnt/data
```

3.B-format-hdd-ext4.sh — SH

```
# 3.B — Format HDD 500GB sebagai ext4 (JIKA belum diformat)
# HATI-HATI: pastikan /dev/sdb BENAR-BENAR HDD data kosong, bukan disk OS!
# Cek ulang dengan: sudo blkid /dev/sdb1 — harus TIDAK ADA output jika kosong.

# 1) Partisi HDD (single partition, GPT)
sudo parted -s /dev/sdb mklabel gpt
sudo parted -s /dev/sdb mkpart primary ext4 0% 100%

# 2) Format ext4 (pilihan terbaik untuk Linux file server — journaling, ACL support)
sudo mkfs.ext4 -L lawu-data -m 1 -T largefile4 /dev/sdb1
#   -L lawu-data  = label disk (memudah identifikasi)
#   -m 1          = reserved 1% untuk root (default 5% terlalu boros di 500GB)
#   -T largefile4 = optimasi untuk file besar (ebook PDF/EPUB, index Tantivy)

# 3) Verifikasi format
sudo blkid /dev/sdb1
# Harus cetak: TYPE="ext4" LABEL="lawu-data"
```

3.C-fstab-mount.sh SH

```
# 3.C — Mount otomatis via /etc/fstab (persistent across reboot)

# 1) Dapat UUID partisi HDD
UUID=$(sudo blkid -s UUID -o value /dev/sdb1)
echo "UUID HDD: $UUID"

# 2) Backup fstab lama
sudo cp /etc/fstab /etc/fstab.bak.$(date +%Y%m%d)

# 3) Tambah entry mount
echo "UUID=$UUID  /mnt/data  ext4  defaults,noatime,nodiratime,data=writeback  0  2" \
  | sudo tee -a /etc/fstab

#   noatime       = jangan update access time (hemat write SSD/HDD, cepat)
#   data=writeback = mode journaling paling cepat (risiko minor korupsi jika power loss)
#                    AMAN karena ada backup harian (Langkah 15)
#
#   UNTUK PRODUKSI SENSITIF: ganti data=writeback → data=ordered (default, lebih aman)

# 4) Mount sekarang
sudo mount -a

# 5) Verifikasi mount
mount | grep /mnt/data
df -hT /mnt/data
# Harus cetak: /dev/sdb1 ext4  492G  ...  /mnt/data
```

3.D-ssd-tuning.sh SH

```
# 3.D — Tuning SSD 128GB (OS disk) untuk umur panjang
# TRIM otomatis + scheduler I/O yang sesuai untuk SSD

# 1) Aktifkan TRIM periodik (hemat SSD, hapus blok tidak terpakai)
sudo systemctl enable --now fstrim.timer
sudo systemctl status fstrim.timer --no-pager

# 2) Set I/O scheduler ke 'mq-deadline' atau 'none' untuk SSD/NVMe
#    (cek yang tersedia dulu)
cat /sys/block/sda/queue/scheduler
# Set persisten via udev:
echo 'ACTION=="add|change", KERNEL=="sd[a-z]", ATTR{queue/rotational}=="0", ATTR{queue/scheduler}="mq-deadline"' \
  | sudo tee /etc/udev/rules.d/60-ssd-scheduler.rules
sudo udevadm control --reload-rules

# 3) (OPSIONAL) Kurangi swappiness khusus SSD (ditimpa Langkah 6 sysctl)
# SSD 128GB cepat penuh jika swap aktif — swap harus di HDD atau disabled
sudo swapoff --all 2>/dev/null || true
sudo sed -i '/swap/d' /etc/fstab  # hapus entry swap lama
```

4 Firewall (UFW) & SSH Hardening

Default deny incoming, hanya buka port 22 (SSH), 80 (HTTP), 443 (HTTPS). Disable SSH password auth, wajib SSH key. Install fail2ban untuk auto-ban IP brute-force.

> ⚠ **Jangan terkunci dari server!**
>
> Sebelum enable UFW & disable SSH password, PASTIKAN Anda sudah setup SSH key (Langkah 4.C) DAN test login dengan key di terminal terpisah. Jika gagal, Anda bisa kembalikan config dari sesi yang masih hidup.

4.A-ufw-firewall.sh — SH

```
# 4.A — Konfigurasi firewall UFW (default deny, hanya buka port yang perlu)
# Paten 4: Linux only — ufw = front-end iptables/nftables (Debian default)

# 1) Reset UFW ke kondisi default (bersih)
sudo ufw --force reset

# 2) Default policy: tolak incoming, izinkan outgoing
sudo ufw default deny incoming
sudo ufw default allow outgoing

# 3) Izinkan SSH (port 22) — WAJIB sebelum enable, kalau tidak Anda terkunci!
sudo ufw allow 22/tcp comment 'SSH admin'

# 4) Izinkan HTTP (port 80) & HTTPS (port 443) untuk Nginx reverse proxy (Langkah 17)
sudo ufw allow 80/tcp comment 'Nginx HTTP'
sudo ufw allow 443/tcp comment 'Nginx HTTPS'

# 5) (OPSIONAL) Batasi akses internal-only untuk port inference & API
#    Hanya izinkan dari localhost (Axum & llama.cpp sudah bind 127.0.0.1, jadi tidak perlu)
#    JANGAN buka 8080/8081 ke publik — hanya via Nginx reverse proxy

# 6) Enable UFW
sudo ufw --force enable

# 7) Verifikasi status
sudo ufw status verbose
sudo ufw status numbered
```

4.B-sshd-hardening.sh — SH

```
# 4.B — SSH hardening (cegah brute-force & privilege escalation)

# 1) Backup config SSH lama
sudo cp /etc/ssh/sshd_config /etc/ssh/sshd_config.bak.$(date +%Y%m%d)

# 2) Edit /etc/ssh/sshd_config — tambah/baris berikut (di akhir file):
sudo tee -a /etc/ssh/sshd_config > /dev/null <<'SSHCONF'

# ===== Lawu-2.5-Fine SSH Hardening =====
PermitRootLogin no              # root tidak boleh SSH langsung
PasswordAuthentication no       # WAJIB pakai SSH key, bukan password
PubkeyAuthentication yes
KbdInteractiveAuthentication no
ChallengeResponseAuthentication no
X11Forwarding no               # tidak perlu X11 di server
AllowTcpForwarding no          # cegah SSH tunneling abuse
AllowAgentForwarding no
ClientAliveInterval 300        # disconnect idle setelah 5 menit
ClientAliveCountMax 2          # 2x ping = 10 menit max idle
MaxAuthTries 3                 # max 3 percobaan login
LoginGraceTime 30              # timeout login 30 detik
SSHCONF

# 3) Restart SSH daemon
sudo systemctl restart sshd
sudo systemctl status sshd --no-pager

# 4) SEBELUM tutup sesi ini: BUKA TERMINAL BARU & test login SSH dengan key
#    Jika gagal login, Anda bisa kembalikan config backup dari sesi yang masih hidup:
#    sudo cp /etc/ssh/sshd_config.bak.* /etc/ssh/sshd_config && sudo systemctl restart sshd
```

4.C-ssh-key-setup.sh SH

```
# 4.C — Generate SSH key pair (di mesin admin/laptop Anda, BUKAN di server)

# 1) Di laptop admin Anda — generate ed25519 key (modern, aman, ringan)
ssh-keygen -t ed25519 -a 100 -C "admin@lawu-server" -f ~/.ssh/lawu_admin

# 2) Copy public key ke server lawu (masukkan password sekali, setelah ini pakai key)
ssh-copy-id -i ~/.ssh/lawu_admin.pub lawu@<IP-SERVER>

# 3) Test login dengan key (tanpa password)
ssh -i ~/.ssh/lawu_admin lawu@<IP-SERVER>

# 4) Setelah login dengan key berhasil, password login otomatis disabled (Langkah 4.B)
#    Verifikasi: SSH dengan password harus DITOLAK
ssh lawu@<IP-SERVER>   # harus gagal: Permission denied (publickey)
```

4.D-fail2ban-sshd.sh SH

```
# 4.D — Install & konfigurasi fail2ban (auto-ban IP setelah 3x gagal SSH)

# 1) fail2ban sudah diinstall di Langkah 1.B — konfigurasi sekarang
sudo tee /etc/fail2ban/jail.d/lawu-sshd.local > /dev/null <<'F2B'
[sshd]
enabled  = true
port     = 22
filter   = sshd
backend  = systemd
maxretry = 3
findtime = 600
bantime  = 3600
ignoreip = 127.0.0.1/8 ::1
F2B

# 2) Restart fail2ban
sudo systemctl restart fail2ban
sudo systemctl enable fail2ban

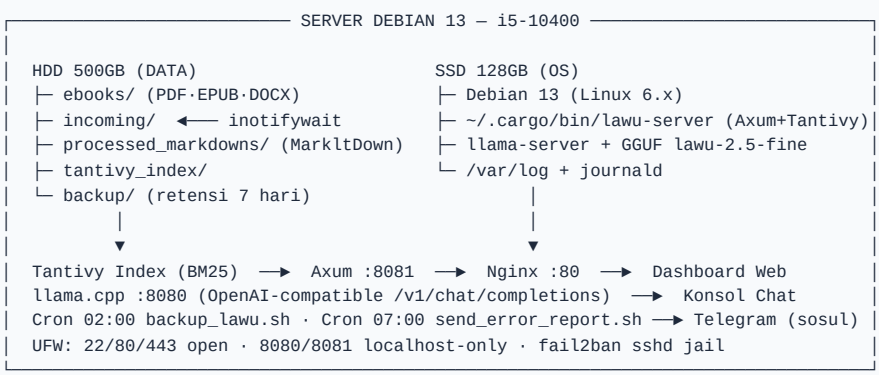
# 3) Verifikasi jail aktif
sudo fail2ban-client status sshd

# 4) (Lewat waktu) Cek IP yang di-ban
sudo fail2ban-client status sshd
# Untuk unban: sudo fail2ban-client set sshd unbanip <IP>
```

5 Ringkasan Arsitektur

Alur data end-to-end dari HDD koleksi ebook hingga dashboard web, termasuk firewall & binding localhost-only.

arsitektur.txt TXT



6 Spesifikasi Infrastruktur & Kernel Tuning (sysctl)

Parameter kernel di /etc/sysctl.d/99-lawu-fine.conf + limits.conf untuk file descriptor besar (Axum + Tantivy).

6.A-apply-sysctl.sh SH

```
# 6.A — Tulis config sysctl Lawu ke file (heredoc — LANGSUNG JALANKAN)
# >>> RUN AS: user admin Anda (yang punya sudo) <<<
# JANGAN jalankan sebagai user lawu — akan error "not in sudoers file"
# Ini CONFIG FILE, bukan script. Heredoc menulis ke /etc/sysctl.d/99-lawu-fine.conf lalu apply.
sudo tee /etc/sysctl.d/99-lawu-fine.conf > /dev/null <<'SYSCTL'
# Tuning kernel untuk search engine ebook Lawu-2.5-Fine

# Manfaatkan RAM 8GB untuk caching disk/Tantivy
# RAM 8GB = mepet untuk 7B Q4_K_M, swappiness 40 (lebih agresif swap ke HDD)
# RAM 16GB+ = swappiness 10 (jarang swap, cache di RAM)
vm.swappiness = 40

# Tulis data kotor ke disk bertahap — cegah I/O stall saat indexing
vm.dirty_background_ratio = 5
vm.dirty_ratio = 15

# Batas file descriptor besar untuk Axum & Tantivy (buka banyak segmen index)
fs.file-max = 2097152

# Pemantauan inotifywait hingga ratusan ribu file ebook di HDD
fs.inotify.max_user_watches = 524288
fs.inotify.max_user_instances = 512

# Perbesar antrean backlog koneksi masuk untuk HTTP API Axum
net.core.somaxconn = 65535
net.ipv4.tcp_max_syn_backlog = 65535

# Tweak TCP untuk API responsiveness
net.ipv4.tcp_fastopen = 3
net.core.netdev_max_backlog = 16384

# Cegah SYN flood (DDoS ringan)
net.ipv4.tcp_syncookies = 1
SYSCTL

# Apply config ke kernel yang sedang berjalan (full path — anti PATH issue)
sudo /usr/sbin/sysctl --system

# Verifikasi nilai sudah berubah (pakai sudo — secure_path include /usr/sbin)
sudo sysctl vm.swappiness fs.file-max fs.inotify.max_user_watches net.core.somaxconn
```

6.B-apply-limits.sh SH

```
# 6.B — Tulis limits.conf untuk user lawu (heredoc — LANGSUNG JALANKAN)
# Naikkan batas file descriptor untuk Axum + Tantivy
sudo tee /etc/security/limits.d/99-lawu-fine.conf > /dev/null <<'LIMITS'
lawu  soft  nofile  1048576
lawu  hard  nofile  2097152
lawu  soft  nproc   65535
lawu  hard  nproc   131072
root  soft  nofile  1048576
root  hard  nofile  2097152
LIMITS

# Aktifkan: logout + login ulang (atau reboot) agar limit baru berlaku
# Verifikasi setelah relogin: ulimit -n  (harus 1048576)
sudo systemctl daemon-reload
```

7 Install Rust Toolchain

Install Rust via rustup (stable + rustfmt + clippy), set RUSTFLAGS target-cpu=native untuk optimasi AVX2 di i5-10400. Verifikasi murni CPU — tidak ada dependency GPU.

7.A-install-rustup.sh SH

```
# 7.A — Install Rust via rustup (official installer, open source Apache-2.0/MIT)
# Paten 1: Rust = backend tunggal. Paten 5: rustup = open source.
# >>> RUN AS: user lawu (TANPA sudo — build tidak butuh root) <<<
# Dari shell user admin Anda (yang punya sudo), switch ke lawu dulu:

# 1) Switch ke user lawu (dari shell user admin Anda)
sudo su - lawu
# Sekarang prompt berubah: lawu@hostname:~$
# Verifikasi Anda sebagai lawu (BUKAN sudo — lawu tidak punya sudo):
whoami    # harus cetak: lawu
id        # groups harus TIDAK ada sudo

# 2) Install rustup (rust compiler + cargo + toolchain manager)
#    Profile 'default' SUDAH include rustfmt + clippy otomatis — tidak perlu --component.
#    (Syntax --component rustfmt clippy SALAH; pakai -c rustfmt,clippy jika ingin eksplisit)
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh -s -- -y \
  --default-toolchain stable \
  --profile default

# 3) Source environment cargo
source ~/.cargo/env

# 4) Verifikasi instalasi
rustc --version
cargo --version
rustup --version

# 5) Verifikasi rustfmt + clippy (sudah include dari profile default)
rustup component list --installed | grep -E 'rustfmt|clippy'
# Harus cetak: clippy-x86_64-unknown-linux-gnu + rustfmt-x86_64-unknown-linux-gnu

# 6) Persist PATH cargo + RUSTFLAGS ke ~/.bashrc
echo 'source ~/.cargo/env' >> ~/.bashrc
echo 'export RUSTFLAGS="-C target-cpu=native"' >> ~/.bashrc
source ~/.bashrc
```

7.B-verify-rust-cpu.sh SH

```
# 7.B — Verifikasi Rust toolchain dengan build hello-world
# >>> RUN AS: user lawu (masih sebagai lawu dari Langkah 7.A, TANPA sudo) <<<
# JANGAN jalankan 'sudo su - lawu' di sini — Anda SUDAH sebagai lawu dari 7.A.
# Jika Anda sudah exit dari lawu, login ulang: 'sudo su - lawu' dari user admin.

# 1) Verifikasi Anda sebagai lawu
whoami    # harus cetak: lawu

# 2) Source cargo env (wajib setiap shell baru — atau tambah ke ~/.bashrc)
source ~/.cargo/env

# Jika 'source ~/.cargo/env' gagal → Langkah 7.A belum dijalankan.
# KEMBALI ke 7.A: curl https://sh.rustup.rs | sh ...
# Jika ~/.cargo/env tidak ada setelah 7.A → rustup install gagal,
# cek log error di 7.A (mungkin butuh build-essential/gcc di Langkah 1.B)

# 3) Buat project dummy
cd /tmp
cargo new hello_lawu --bin
cd hello_lawu

# 4) Build dengan optimasi native CPU (AVX2 di i5-10400)
RUSTFLAGS="-C target-cpu=native" cargo build --release
./target/release/hello_lawu
# Harus cetak: Hello, world!

# 5) Cek binary ter-link dengan benar (Paten 3: murni CPU, tanpa GPU dep)
ldd ./target/release/hello_lawu
# Harus HANYA cetak: libc.so.6, libgcc_s.so.1, ld-linux-x86-64.so.2
# JANGAN ada libcuda, libnvrtc, atau libGPU apa pun

# 6) (Optional) Persist RUSTFLAGS + cargo PATH ke ~/.bashrc
echo 'source ~/.cargo/env' >> ~/.bashrc
echo 'export RUSTFLAGS="-C target-cpu=native"' >> ~/.bashrc
# Sekarang setiap shell baru otomatis pakai cargo + RUSTFLAGS

# 7) Cleanup
cd / && rm -rf /tmp/hello_lawu
```

8 Proyek Cargo — lawu-server

Inisialisasi project Cargo, paste Cargo.toml + main.rs + schema.rs, build release dengan optimasi native CPU, install binary ke ~/.cargo/bin/.

8.A-cargo-init.sh SH

```sh
# 8.A — Inisialisasi project Cargo lawu-server
# >>> RUN AS: user lawu (masih sebagai lawu dari Langkah 7, TANPA sudo) <<<
# JANGAN jalankan 'sudo su - lawu' di sini — Anda SUDAH sebagai lawu dari Langkah 7.
# Jika Anda sudah exit dari lawu, login ulang: 'sudo su - lawu' dari user admin.

# 1) Verifikasi Anda sebagai lawu + cargo di PATH
whoami                    # harus cetak: lawu
source ~/.cargo/env       # load cargo env (atau buka shell baru)
cargo --version           # harus cetak: cargo 1.x.x

# 2) Buat project di home directory
cd ~
cargo new --bin lawu-server
cd lawu-server

# 3) Cek struktur direktori yang dihasilkan (pakai ls — 'tree' jarang terinstall)
ls -la
ls -la src/
# Output yang diharapkan:
# ./:
#   drwxr-xr-x  src/
#   -rw-r--r--  Cargo.toml
#   -rw-r--r--  .gitignore
# ./src/:
#   -rw-r--r--  main.rs

# (OPSIONAL) Jika ingin pakai 'tree' untuk visualisasi hierarki:
#   sudo apt install tree   (dari user admin, bukan lawu)
#   tree -L 2
# Tapi 'ls -la' + 'ls -la src/' di atas sudah cukup.
```

~/lawu-server/Cargo.toml TOML

```toml
# ~/lawu-server/Cargo.toml
# Compatible dengan Rust 1.99 (2026) — gunakan axum 0.8 + tantivy 0.24
[package]
name = "lawu-server"
version = "0.1.0"
edition = "2021"
authors = ["Lawu Team <lawu@example.id>"]
description = "Lawu-2.5-Fine search API — Axum + Tantivy"
license = "MIT"

[dependencies]
axum = "0.8"
tokio = { version = "1", features = ["full"] }
tantivy = "0.24"
serde = { version = "1", features = ["derive"] }
serde_json = "1"
tracing = "0.1"
tracing-subscriber = { version = "0.3", features = ["env-filter"] }
reqwest = { version = "0.12", features = ["json"] }
walkdir = "2"
rayon = "1"
anyhow = "1"
thiserror = "2"
tower-http = { version = "0.6", features = ["cors", "trace"] }
chrono = { version = "0.4", features = ["serde"] }

[profile.release]
opt-level = 3
lto = true
codegen-units = 1
strip = true        # strip symbol → binary lebih kecil
panic = "abort"     # hemat binary size, tanpa unwind tables
```

8.B-build-release.sh SH

```sh
# 8.B — Build lawu-server (release mode, optimasi native CPU)

# Sebagai user lawu, di ~/lawu-server/
source ~/.cargo/env
cd ~/lawu-server

# 1) Copy Cargo.toml dari Langkah 8.A ke file (paste via editor)
nano Cargo.toml
# (paste konten, Ctrl+O, Enter, Ctrl+X)

# 2) Copy src/main.rs dari Langkah 10 ke file
nano src/main.rs
# (paste konten, Ctrl+O, Enter, Ctrl+X)

# 3) Copy src/schema.rs dari Langkah 9 ke file
mkdir -p src
nano src/schema.rs
# (paste konten, Ctrl+O, Enter, Ctrl+X)

# 4) Build release dengan optimasi native CPU
RUSTFLAGS="-C target-cpu=native" cargo build --release

# 5) Install binary ke ~/.cargo/bin/ (akses global dari PATH)
cargo install --path .
# Binary tersimpan di: ~/.cargo/bin/lawu-server

# 6) Verifikasi binary ter-link bersih (Paten 3: murni CPU)
ldd ~/.cargo/bin/lawu-server
# Harus HANYA: libc, libgcc_s, libm, libpthread, libdl, ld-linux
# JANGAN ada libcuda / libnvrtc / libGPU
```

9 Skema Full-Text Tantivy

Skema multi-field (title, body, path, file_type, category, pages, size_bytes, indexed_at) dengan BM25 scoring, FAST field untuk sort numerik.

~/lawu-server/src/schema.rs RS

```rs
// ~/lawu-server/src/schema.rs
// Compatible dengan Tantivy 0.24 + Rust 1.99 (2026)
use tantivy::schema::{Schema, TEXT, STORED, FAST};

pub fn build_schema() -> Schema {
    // Skema multi-field — identik dengan kontrak dashboard:
    // title, body, path, file_type, category, pages, size_bytes
    let mut schema = Schema::builder();

    // title — STORED untuk di-return di hasil pencarian, TEXT untuk BM25
    schema.add_text_field("title", TEXT | STORED);

    // body — TEXT only (tidak di-store, hemat disk; snippet di-generate saat query)
    schema.add_text_field("body", TEXT);

    // path — unique per dokumen, STORED untuk lookup file asli
    schema.add_text_field("path", TEXT | STORED);

    // file_type — PDF | EPUB | DOCX (filter TermQuery presisi)
    schema.add_text_field("file_type", TEXT);

    // category — misal "Agama Islam", "Programming", "AI/ML"
    schema.add_text_field("category", TEXT);

    // pages — jumlah halaman (FAST untuk sort/filter numerik)
    schema.add_u64_field("pages", FAST | STORED);

    // size_bytes — ukuran file asli (FAST untuk sort)
    schema.add_u64_field("size_bytes", FAST | STORED);

    // NOTE: indexed_at (date field) di-add nanti saat butuh sort by timestamp.
    // Tantivy 0.24 add_date_field butuh DateOptions type annotation — skip untuk now
    // agar schema.rs pasti compile. Search by title/body BM25 sudah cukup untuk MVP.

    schema.build()
}

// Index writer dengan commit per batch (Rayon memparalelkan ekstraksi):
// let writer = index.writer(64_000_000)?;            // 64 MB heap
// writer.add_document(doc)?;
// writer.commit()?;  // posting list BM25 siap dicari
```

10 Endpoint API Backend (Axum)

GET /api/stats · POST /api/search (QueryParser + TopDocs) · POST /api/telegram/test. Bind 127.0.0.1:8081 (akses via Nginx reverse proxy).

~/lawu-server/src/main.rs RS

```rust
// ~/lawu-server/src/main.rs
// Compatible dengan Axum 0.8 + Tantivy 0.24 + Rust 1.99 (2026)
use axum::{
    extract::State,
    http::StatusCode,
    routing::{get, post},
    Json, Router,
};
use serde::{Deserialize, Serialize};
use std::sync::Arc;
use tantivy::collector::TopDocs;
use tantivy::query::QueryParser;
use tantivy::schema::Value;
use tantivy::{Index, IndexReader, ReloadPolicy};

// Module declaration — schema.rs harus ada di src/schema.rs (Langkah 9)
mod schema;

#[derive(Clone)]
struct AppState {
    index: Arc<Index>,
    reader: Arc<IndexReader>,
    schema_fields: Arc<SchemaFields>,
}

struct SchemaFields {
    title: tantivy::schema::Field,
    body: tantivy::schema::Field,
    path: tantivy::schema::Field,
    file_type: tantivy::schema::Field,
    category: tantivy::schema::Field,
}

#[derive(Deserialize)]
struct SearchRequest {
    query: String,
    file_type: Option<String>,
    category: Option<String>,
    limit: Option<usize>,
}

#[derive(Serialize)]
struct SearchHit {
    title: String,
    path: String,
    file_type: String,
    category: String,
    score: f32,
}

#[derive(Serialize)]
struct SearchResponse {
    results: Vec<SearchHit>,
    total: usize,
}

#[tokio::main]
async fn main() -> anyhow::Result<()> {
    tracing_subscriber::fmt().with_env_filter("info").init();

    // 1) Buka index Tantivy di HDD, atau buat baru jika belum ada
    let index_path = "/mnt/data/tantivy_index";
    let index = Index::open_in_dir(index_path).unwrap_or_else(|_| {
        // Index belum ada — buat baru dengan schema
        std::fs::create_dir_all(index_path).expect("buat dir tantivy_index gagal");
        let schema = schema::build_schema();
        Index::create_in_dir(index_path, schema).expect("create tantivy index gagal")
    });

    // 2) Reader dengan auto-reload (indexer & searcher di proses yang sama)
    let reader = index
        .reader_builder()
        .reload_policy(ReloadPolicy::OnCommitWithDelay)
        .try_into()?;

    // 3) Cache field handles (lebih cepat daripada get_field di setiap request)
    let schema = index.schema();
    let schema_fields = Arc::new(SchemaFields {
        title: schema.get_field("title").expect("field title tidak ada"),
        body: schema.get_field("body").expect("field body tidak ada"),
        path: schema.get_field("path").expect("field path tidak ada"),
```

```
        file_type: schema.get_field("file_type").expect("field file_type tidak ada"),
        category: schema.get_field("category").expect("field category tidak ada"),
    });

    let state = AppState {
        index: Arc::new(index),
        reader: Arc::new(reader),
        schema_fields,
    };

    // 4) Router Axum 0.8 — bind 127.0.0.1:8081 (localhost only, via Nginx reverse proxy)
    let app = Router::new()
        .route("/api/stats", get(stats_handler))
        .route("/api/search", post(search_handler))
        .route("/api/telegram/test", post(telegram_test_handler))
        .with_state(state);

    let listener = tokio::net::TcpListener::bind("127.0.0.1:8081").await?;
    tracing::info!("lawu-server berjalan di http://127.0.0.1:8081");
    axum::serve(listener, app).await?;
    Ok(())
}

async fn search_handler(
    State(state): State<AppState>,
    Json(req): Json<SearchRequest>,
) -> Result<Json<SearchResponse>, StatusCode> {
    let searcher = state.reader.searcher();
    let f = &state.schema_fields;

    // QueryParser: cari di title + body (BM25 default)
    let query_parser = QueryParser::for_index(
        &state.index,
        vec![f.title, f.body],
    );
    let query = query_parser
        .parse_query(&req.query)
        .map_err(|_| StatusCode::BAD_REQUEST)?;

    let limit = req.limit.unwrap_or(60).min(200);
    let top_docs = searcher
        .search(&query, &TopDocs::with_limit(limit))
        .map_err(|_| StatusCode::INTERNAL_SERVER_ERROR)?;

    let hits: Vec<SearchHit> = top_docs
        .into_iter()
        .map(|(score, doc_addr)| {
            let doc: tantivy::TantivyDocument = searcher.doc(doc_addr).unwrap_or_default();
            SearchHit {
                title: doc.get_first(f.title).and_then(|v| v.as_str()).unwrap_or("").to_string(),
                path: doc.get_first(f.path).and_then(|v| v.as_str()).unwrap_or("").to_string(),
                file_type: doc.get_first(f.file_type).and_then(|v| v.as_str()).unwrap_or("").to_string(),
                category: doc.get_first(f.category).and_then(|v| v.as_str()).unwrap_or("").to_string(),
                score,
            }
        })
        .collect();

    Ok(Json(SearchResponse { total: hits.len(), results: hits }))
}

async fn stats_handler(State(state): State<AppState>) -> Json<serde_json::Value> {
    // Return metrik: total docs, index size, dsb.
    let searcher = state.reader.searcher();
    Json(serde_json::json!({
        "docs": {
            "total": searcher.num_docs(),
            "byType": [],
            "byCategory": [],
            "indexSizeMb": 0,
            "lastIndexedAt": null
        },
        "chats": { "conversations": 0, "messages": 0 },
        "alerts": { "total24h": 0 }
    }))
}

async fn telegram_test_handler() -> Json<serde_json::Value> {
    // Tes konektivitas Telegram Bot (mode simulasi — token belum di-set)
    Json(serde_json::json!({
        "ok": true,
        "mode": "simulasi",
        "message": "Telegram belum dikonfigurasi — isi /etc/lawu/env"
    }))
}
```

11 Build llama.cpp (Pure CPU)

Clone repo llama.cpp (MIT license), build dengan GGML_NATIVE=ON + AVX2 + FMA + F16C untuk i5-10400. Verifikasi binary TIDAK ada dependency CUDA/GPU.

11.A-clone-llamacpp.sh SH

```
# 11.A — Clone & build llama.cpp (open source MIT)
# Paten 3: murni CPU. Paten 5: llama.cpp = MIT license.
# llama.cpp TIDAK butuh GPU — pure CPU dengan optimasi AVX2 otomatis di i5-10400.
# >>> RUN AS: user lawu (masih sebagai lawu dari Langkah 10, TANPA sudo) <<<

# 1) Verifikasi sebagai lawu + di home directory
whoami    # harus: lawu
cd ~
mkdir -p ~/src
cd ~/src

# 2) Clone repo llama.cpp (official ggml-org — repo sudah di-migrate dari ggerganov)
git clone https://github.com/ggml-org/llama.cpp.git
cd llama.cpp

# 3) Cek 5 tag release terbaru, pilih yang paling baru (stabil)
git tag | grep -E '^b[0-9]+$' | sort -V | tail -5
# Contoh output Nov 2026: b12065, b12066, b12068, b12069
# Pilih yang TERAKHIR (paling baru stabil)

# 4) Checkout tag stabil terbaru
git checkout b12069   # ganti dengan tag terbaru yang muncul di output step 3

# 5) Update submodule (jika ada — .gitmodules mungkin kosong, aman)
git submodule update --init --recursive

# 6) Verifikasi LICENSE (MIT — Paten 5)
head -3 LICENSE
# Harus cetak: MIT License + copyright holder
```

11.B-build-cmake.sh SH

```sh
# 11.B — Build llama.cpp dengan static linking + optimasi CPU native (AVX2 i5-10400)
# >>> RUN AS: user lawu (masih sebagai lawu dari Langkah 11.A, TANPA sudo) <<<
# Static linking = binary mandiri, tidak butuh .so eksternal → mudah untuk systemd

cd ~/src/llama.cpp

# 1) Konfigurasi build dengan CMake (STATIC LINKING)
#    GGML_NATIVE=ON = deteksi otomatis AVX2/AVX512/FMA di i5-10400
#    LLAMA_BUILD_SERVER=ON = bangun 'llama-server' (OpenAI-compatible endpoint)
#    BUILD_SHARED_LIBS=OFF + LLAMA_STATIC=ON = binary mandiri, tidak butuh .so
cmake -B build \
  -DGGML_NATIVE=ON \
  -DGGML_AVX2=ON \
  -DGGML_FMA=ON \
  -DGGML_F16C=ON \
  -DLLAMA_BUILD_SERVER=ON \
  -DLLAMA_CURL=ON \
  -DCMAKE_BUILD_TYPE=Release \
  -DBUILD_SHARED_LIBS=OFF \
  -DLLAMA_STATIC=ON \
  -G Ninja

# 2) Build paralel — GUNAKAN -j 3 untuk RAM 8GB (jangan -j 6, OOM!)
#    Estimasi: 10-15 menit dengan 3 thread (RAM 8GB aman)
cmake --build build --config Release -j 3

# ⚠️ RAM 8GB BUILD RULE:
#   -j 3 = aman (3 × ~1GB RAM = 3GB, sisa 5GB untuk OS)
#   -j 4 = mepet (4 × ~1GB = 4GB, bisa OOM kalau OS sibuk)
#   -j 6 = OOM (6 × ~1GB = 6GB, RAM 8GB tidak cukup)

# 3) Verifikasi binary ada + size besar (static linking → 15-25MB, bukan 18KB)
ls -lh build/bin/llama-server
ls -lh build/bin/llama-quantize
ls -lh build/bin/llama-cli

# 4) Verifikasi STATIC LINKING — hanya dependency sistem (NO .so dari llama.cpp)
ldd build/bin/llama-server
# Harus HANYA cetak: libstdc++, libgcc_s, libc, libm, libdl, libpthread, ld-linux
# TIDAK boleh ada: libllama.so, libggml.so, libllama-server-impl.so (sudah static)

# 5) Verifikasi murni CPU (Paten 3) — TIDAK ada dependency CUDA/GPU
ldd build/bin/llama-server | grep -iE 'cuda|nvrtc|cudnn|libgpu' \
  && echo "✗ ADA GPU dep! Paten 3 gagal" \
  || echo "✓ Murni CPU (Paten 3 terpenuhi)"

# 6) Test binary bisa run + cetak version
~/src/llama.cpp/build/bin/llama-server --version
# Harus cetak: version: b12069 (atau tag yang Anda checkout)

# Estimasi waktu build: 10-15 menit di i5-10400 dengan 3 thread (RAM 8GB)
# Jika masih OOM (jarang), turunkan -j 3 → -j 2
```

12 Konversi & Kuantisasi Model GGUF

Konversi HuggingFace → GGUF F16, lalu kuantisasi ke Q4_K_M (4.4 GB untuk model 7B). Verifikasi metadata GGUF.

12.A-convert-hf-to-gguf.sh SH

```
# 12.A — Dapatkan model GGUF (PILIH SALAH SATU OPDI)
# >>> RUN AS: user lawu (masih sebagai lawu dari Langkah 11, TANPA sudo) <<<
#
# PENTING: ~/models/ adalah SYMLINK ke /mnt/data/models/ (HDD).
# Semua file model fisik disimpan di HDD (3.6GB+), bukan SSD 128GB.
# Verifikasi symlink:
ls -la ~/models
# Harus cetak: ~/models -> /mnt/data/models
df -h /mnt/data   # pastikan HDD ada space 5GB+

# ═══════════════════════════════════════════════════════════
# OPSI 1 (RECOMMENDED — CEPAT): Download Pre-quantized GGUF Q3_K_M
# ═══════════════════════════════════════════════════════════
# Langsung download Q3_K_M yang sudah di-quantize komunitas (bartowski).
# Hemat disk (3.6GB vs 14GB HF + 14GB F16), hemat waktu (skip convert step).

cd ~/models   # = /mnt/data/models via symlink

# Download pakai wget (TIDAK butuh venv/pip/huggingface-cli)
# -c = resume kalau koneksi putus
# Estimasi: 3.6GB, 5-30 menit tergantung bandwidth
wget -c https://huggingface.co/bartowski/Qwen2.5-Coder-7B-Instruct-GGUF/resolve/main/Qwen2.5-Coder-7B-Instruct-Q3_K_M.gguf \
  -O ~/models/lawu-2.5-fine-q3_k_m.gguf

# Verifikasi file di HDD (bukan SSD!)
ls -lh ~/models/lawu-2.5-fine-q3_k_m.gguf
ls -lh /mnt/data/models/lawu-2.5-fine-q3_k_m.gguf   # path absolut HDD
df -h /mnt/data   # HDD usage naik ~3.6GB

# File: GGUF (binary). Test bisa dibaca llama-server:
~/src/llama.cpp/build/bin/llama-server --version   # pastikan binary OK

# SKIP ke Langkah 13 (serve) — tidak perlu quantize (sudah Q3_K_M dari bartowski)

# ═══════════════════════════════════════════════════════════
# OPSI 2 (LENGKAP): Convert HuggingFace → GGUF F16 → Q3_K_M sendiri
# ═══════════════════════════════════════════════════════════
# Pilih opsi ini jika Anda punya model fine-tune Lawu sendiri,
# atau mau proses lengkap untuk dokumentasi.

# 12.A.1 — Download HF model (14GB, butuh waktu lama)
cd ~/models
git lfs install   # pastikan git-lfs aktif
git clone https://huggingface.co/Qwen/Qwen2.5-Coder-7B-Instruct \
  ~/models/lawu-2.5-fine-hf
# Estimasi: 14GB, 10-30 menit tergantung bandwidth

# 12.A.2 — Buat venv Python (jika belum ada dari Langkah 1.C)
python3 -m venv ~/venvs/lawu
source ~/venvs/lawu/bin/activate

# 12.A.3 — Install llama.cpp Python deps
cd ~/src/llama.cpp
pip install -r requirements.txt

# 12.A.4 — Convert HF → GGUF F16 (full precision, ukuran besar)
python3 convert_hf_to_gguf.py ~/models/lawu-2.5-fine-hf \
  --outfile ~/models/lawu-2.5-fine-f16.gguf \
  --outtype f16
deactivate

# 12.A.5 — Verifikasi GGUF F16 (di HDD via symlink)
ls -lh ~/models/lawu-2.5-fine-f16.gguf
# Estimasi: 7B F16 ≈ 14 GB

# 12.A.6 — Cek metadata GGUF (opsional, jika binary llama-gguf-dump ada)
~/src/llama.cpp/build/bin/llama-gguf-dump ~/models/lawu-2.5-fine-f16.gguf | head -20

# Lanjut ke Langkah 12.B (quantize F16 → Q3_K_M)
```

12.B-quantize-q4_k_m.sh SH

```sh
# 12.B — Kuantisasi GGUF F16 → Q3_K_M (RAM 8GB tuning untuk 7B model)
# >>> RUN AS: user lawu (masih sebagai lawu dari Langkah 12.A, TANPA sudo) <<<
# RAM 8GB: pakai Q3_K_M (3.6GB) — Q4_K_M (4.4GB) terlalu mepet

cd ~/src/llama.cpp

# 1) Kuantisasi ke Q3_K_M (3-bit, K-quant medium) — RAM 8GB optimal
#    Trade-off: ukuran ~3.6GB, kualitas sedikit turun dari Q4_K_M, throughput tinggi
#    RAM 8GB budget: 3.6GB model + 0.5GB KV + 2.5GB OS + 1.4GB Axum/Tantivy = ~8GB (safe)
./build/bin/llama-quantize \
  ~/models/lawu-2.5-fine-f16.gguf \
  ~/models/lawu-2.5-fine-q3_k_m.gguf Q3_K_M

# 2) Verifikasi hasil kuantisasi
ls -lh ~/models/lawu-2.5-fine-q3_k_m.gguf
# Estimasi: 7B Q3_K_M ≈ 3.6 GB

# 3) (OPSIONAL) Jika RAM upgrade ke 16GB nanti, re-quantize ke Q4_K_M (kualitas lebih baik)
#    ATAU jika mau test Q4_K_M sekarang (RAM mepet, bisa OOM saat inference):
#    ./build/bin/llama-quantize \
#      ~/models/lawu-2.5-fine-f16.gguf \
#      ~/models/lawu-2.5-fine-q4_k_m.gguf Q4_K_M
#    Q4_K_M ≈ 4.4 GB — throughput ~7-11 tok/s, BISA OOM kalau prompt panjang

# 4) Update nama file model di Langkah 13 (LLAMA_SERVE) & Langkah 14 (SYSTEMD_LLAMA)
#    Ganti 'lawu-2.5-fine-q4_k_m.gguf' → 'lawu-2.5-fine-q3_k_m.gguf'
#    (Panduan web sudah otomatis pakai q3_k_m, tapi systemd service file perlu update manual)

# 5) Hapus GGUF F16 setelah quantize sukses (hemat HDD 500GB)
#    SIMPAN F16 jika Anda berencana re-quantize ke Q4_K_M/Q5_K_M nanti (saat RAM upgrade)
rm -f ~/models/lawu-2.5-fine-f16.gguf

# 6) Verifikasi akhir model — cek metadata GGUF
~/src/llama.cpp/build/bin/llama-gguf-dump ~/models/lawu-2.5-fine-q3_k_m.gguf | head -30
# Harus cetak: general.architecture = qwen2, general.file_type = Q3_K_M,
#              tokenizer.ggml.model = qwen2, dll.
```

13 Serve llama.cpp (OpenAI-compatible)

Jalankan llama-server dengan -t 6, --mmap, -c 4096, bind 127.0.0.1:8080. Test endpoint /v1/models & /v1/chat/completions. Estimasi 9-14 tok/s di i5-10400.

13.A-serve-llama.sh SH

```
# 13.A — Jalankan llama-server (OpenAI-compatible endpoint)
# >>> RUN AS: user lawu (masih sebagai lawu dari Langkah 12, TANPA sudo) <<<
# RAM 8GB: pakai -t 4 + -c 2048 (bukan -t 6 + -c 4096 — itu untuk RAM 16GB)
# Alternatif: pakai model 3B (Qwen2.5-Coder-3B) untuk headroom lebih besar

cd ~/src/llama.cpp

# 1) Tes serve manual dulu (sebelum systemd di Langkah 14)
#    NOTE: --mmap sudah TIDAK ada di llama.cpp b12069+ (mmap default ON, tidak perlu flag)
./build/bin/llama-server \
  -m ~/models/lawu-2.5-fine-q3_k_m.gguf \
  --alias lawu-2.5-fine \
  -t 4 \
  -c 2048 \
  --host 127.0.0.1 \
  --port 8080

# Parameter penjelasan (RAM 8GB tuning):
#   -m           = path model GGUF
#   --alias      = nama model di endpoint OpenAI-compatible
#   -t 4         = 4 thread (sisakan 2 thread untuk OS + Axum + Tantivy)
#   -c 2048      = context window 2K token (hemat KV cache RAM ~512MB)
#   --host 127.0.0.1 = bind localhost SAJA (akses via Nginx reverse proxy)
#   --port 8080  = port inference
#   (mmap otomatis ON — model load via memory-map, hemat RAM)

# RAM 8GB BUDGET (7B Q3_K_M):
#   Model mmap load   : ~3.6GB
#   KV cache ctx 2048  : ~0.5GB
#   Axum + Tantivy    : ~0.6GB
#   OS + cache         : ~3.3GB
#   TOTAL              : ~8.0GB (aman — ada ~1.3GB headroom untuk page cache)

# 2) Di terminal lain, test endpoint OpenAI-compatible
curl http://127.0.0.1:8080/v1/models | jq

curl http://127.0.0.1:8080/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{
    "model": "lawu-2.5-fine",
    "messages": [{"role": "user", "content": "Halo, siapa kamu?"}],
    "max_tokens": 100
  }' | jq

# 3) Estimasi throughput RAM 8GB (i5-10400, -t 4, 7B model):
#    7B Q3_K_M (3.6GB)  : ~10-15 tok/s (RECOMMENDED — aman, kualitas masih bagus)
#    7B Q4_K_M (4.4GB)  : ~7-11 tok/s (mepet, bisa OOM kalau prompt panjang)
#    3B Q4_K_M (2GB)    : ~25-35 tok/s (headroom terbesar, throughput tertinggi)

# 4) Jika OOM saat inference (kill by kernel), ALTERNATIF:
#    a) Pakai model lebih kecil: Qwen2.5-Coder-3B Q4_K_M (~2GB)
#    b) Turunkan context: -c 2048 → -c 1024
#    c) Aktifkan swap ke HDD (BUKAN SSD): sudo mkswap /dev/sdb2 + swapon
#    d) Kurangi thread: -t 4 → -t 3

# 5) Stop manual serve (Ctrl+C), lanjut setup systemd di Langkah 14
```

14 Unit systemd (Auto-restart & Boot)

Dua service: llama-lawu (inference) & lawu-2.5-fine (API Axum). Hardening systemd (NoNewPrivileges, ProtectSystem, PrivateTmp). Auto-restart on crash.

14.A-write-llama-service.sh — SH

```
# 14.A — Tulis unit file systemd untuk llama.cpp (heredoc — LANGSUNG JALANKAN)
# Service inference llama.cpp (CPU only — Paten 3)
sudo tee /etc/systemd/system/llama-lawu.service > /dev/null <<'UNIT'
[Unit]
Description=Lawu-2.5-Fine — llama.cpp GGUF server (CPU)
After=network.target
After=mnt-data.mount

[Service]
User=lawu
Group=lawu
Type=simple
ExecStart=/home/lawu/src/llama.cpp/build/bin/llama-server -m /home/lawu/models/lawu-2.5-fine-q3_k_m.gguf --alias lawu-2.5-fine -t
Restart=always
RestartSec=3
LimitNOFILE=65535

# Hardening systemd
NoNewPrivileges=yes
ProtectSystem=full
ProtectHome=read-only
PrivateTmp=yes
ReadWritePaths=/mnt/data /home/lawu/models

[Install]
WantedBy=multi-user.target
UNIT

# Verify file created
cat /etc/systemd/system/llama-lawu.service | head -10
```

14.B-write-axum-service.sh — SH

```
# 14.B — Tulis unit file systemd untuk Axum API (heredoc — LANGSUNG JALANKAN)
# Service API search Axum + Tantivy
sudo tee /etc/systemd/system/lawu-2.5-fine.service > /dev/null <<'UNIT'
[Unit]
Description=Lawu-2.5-Fine — Axum + Tantivy search API
After=network.target llama-lawu.service
Requires=llama-lawu.service

[Service]
User=lawu
Group=lawu
Type=simple
Environment=RUST_LOG=info
Environment=LAWU_INDEX_PATH=/mnt/data/tantivy_index
Environment=LAWU_LLM_URL=http://127.0.0.1:8080
EnvironmentFile=/etc/lawu/env
ExecStart=/home/lawu/.cargo/bin/lawu-server
Restart=always
RestartSec=3
LimitNOFILE=65535

# Hardening systemd
NoNewPrivileges=yes
ProtectSystem=full
ProtectHome=read-only
PrivateTmp=yes
ReadWritePaths=/mnt/data /var/log/lawu

[Install]
WantedBy=multi-user.target
UNIT

# Verify file created
cat /etc/systemd/system/lawu-2.5-fine.service | head -10
```

14.C-enable-services.sh SH

```
# 14.B — Aktifkan & enable kedua service systemd

# 1) Buat direktori log (jika belum)
sudo mkdir -p /var/log/lawu
sudo chown lawu:lawu /var/log/lawu

# 2) Tulis unit files (paste SYSTEMD_LLAMA & SYSTEMD_AXUM di atas)
sudo nano /etc/systemd/system/llama-lawu.service
sudo nano /etc/systemd/system/lawu-2.5-fine.service

# 3) Reload systemd daemon
sudo systemctl daemon-reload

# 4) Enable & start service (urutan: llama dulu, baru axum)
sudo systemctl enable --now llama-lawu
sleep 3   # tunggu model load
sudo systemctl enable --now lawu-2.5-fine

# 5) Verifikasi status
sudo systemctl status llama-lawu --no-pager
sudo systemctl status lawu-2.5-fine --no-pager

# 6) Cek log real-time
sudo journalctl -u llama-lawu -f
sudo journalctl -u lawu-2.5-fine -f

# 7) Test endpoint API
curl http://127.0.0.1:8081/api/stats | jq
curl -X POST http://127.0.0.1:8081/api/search \
  -H "Content-Type: application/json" \
  -d '{"query":"test","limit":5}' | jq
```

15 Otomasi Pemantauan & Operasional

File watcher inotifywait, backup harian (retensi 7 hari), laporan error harian via Telegram. Cron jobs & environment file /etc/lawu/env.

/opt/lawu/scripts/watch_ebooks.sh SH

```
# /opt/lawu/scripts/watch_ebooks.sh — File Watcher & Notifikasi Real-time
#!/usr/bin/env bash
# Paten 5: inotifywait = GPL-2 (open source, aman)
set -euo pipefail

WATCH_DIR="/mnt/data/ebooks/incoming"
LOG_FILE="/var/log/lawu/watch.log"
TELEGRAM_TOKEN="${TELEGRAM_TOKEN:-}"   # set via /etc/lawu/env
TELEGRAM_CHAT="${TELEGRAM_CHAT:-}"

send_tg() {
  [ -z "$TELEGRAM_TOKEN" ] && return 0   # mode simulasi jika token kosong
  curl -s -X POST "https://api.telegram.org/bot${TELEGRAM_TOKEN}/sendMessage" \
    -d chat_id="${TELEGRAM_CHAT}" -d text="$1" >/dev/null || true
}

mkdir -p "$(dirname "$LOG_FILE")"

inotifywait -m -e close_write -e moved_to --format '%w%f' "${WATCH_DIR}" |
while read -r FILE; do
  case "${FILE,,}" in
    *.pdf|*.epub|*.docx)
      echo "[$(date '+%F %T')] ${FILE}" >> "${LOG_FILE}"
      # 1) Ekstrak teks (MarkltDown via venv)
      source ~/venvs/lawu/bin/activate
      markitdown "${FILE}" >> /mnt/data/processed_markdowns/index.md
      deactivate
      # 2) Reindex Tantivy (trigger via API Axum) — pakai jq untuk build JSON aman
      PAYLOAD=$(jq -n --arg p "$FILE" '{path:$p}')
      curl -s -X POST http://127.0.0.1:8081/api/reindex \
        -H "Content-Type: application/json" -d "$PAYLOAD" || true
      # 3) Notifikasi Telegram
      send_tg "📚 Ebook baru terindeks: $(basename "${FILE}") — Lawu-2.5-Fine"
      ;;
  esac
done
```

/opt/lawu/scripts/backup_lawu.sh SH

```
# /opt/lawu/scripts/backup_lawu.sh — Backup Harian & Rotasi Retensi 7 Hari
#!/usr/bin/env bash
# Cron 02:00 WIB
set -euo pipefail

STAMP=$(date +%Y%m%d_%H%M)
DEST="/mnt/data/backup"

mkdir -p "${DEST}"

tar -czf "${DEST}/lawu_${STAMP}.tar.gz" \
  /mnt/data/processed_markdowns \
  /mnt/data/tantivy_index

# Rotasi retensi: hapus arsip lebih tua dari 7 hari
find "${DEST}" -name 'lawu_*.tar.gz' -mtime +7 -delete

echo "[$(date '+%F %T')] Backup ${STAMP} selesai" >> /var/log/lawu/backup.log
```

/opt/lawu/scripts/send_error_report.sh SH

```sh
# /opt/lawu/scripts/send_error_report.sh — Monitoring Error Log Harian
#!/usr/bin/env bash
# Cron 07:00 WIB
set -euo pipefail

source /etc/lawu/env   # load TELEGRAM_TOKEN & TELEGRAM_CHAT

NGINX_ERR=$(tail -n 5 /var/log/nginx/error.log 2>/dev/null || echo "kosong")
JOURNAL_ERR=$(journalctl -u lawu-2.5-fine.service -u llama-lawu.service -p err..warning --since "24h ago" \
  --no-pager | tail -n 20 || echo "kosong")

MSG="🛠️ <b>Lawu-2.5-Fine — Laporan Error Harian</b>
📅 $(date '+%F %T WIB')

<b>Nginx error (5 baris terakhir):</b>
```
${NGINX_ERR}
```
<b>Journalctl Lawu (err..warning, 24 jam):</b>
```
${JOURNAL_ERR}
```"

curl -s -X POST "https://api.telegram.org/bot${TELEGRAM_TOKEN}/sendMessage" \
  -d chat_id="${TELEGRAM_CHAT}" \
  --data-urlencode "text=${MSG}" \
  -d parse_mode="HTML" >/dev/null || true
```

/etc/lawu/env SH

```sh
# /etc/lawu/env — Environment variables untuk Telegram (jangan commit ke git!)
# Set permission 640, owner root:lawu
sudo mkdir -p /etc/lawu
sudo tee /etc/lawu/env > /dev/null <<'ENV'
TELEGRAM_TOKEN=123456:ABC-YOUR-TOKEN-FROM-BOTFATHER
TELEGRAM_CHAT=123456789
ENV

sudo chown root:lawu /etc/lawu/env
sudo chmod 640 /etc/lawu/env

# Set environment file dipakai oleh systemd service lawu-2.5-fine:
# Tambahkan baris berikut ke /etc/systemd/system/lawu-2.5-fine.service di [Service]:
#   EnvironmentFile=/etc/lawu/env

# Reload & restart
sudo systemctl daemon-reload
sudo systemctl restart lawu-2.5-fine
```

crontab -e (user lawu) SH

```sh
# crontab -e  (user lawu)
# Backup harian pukul 02:00 WIB (Asia/Jakarta = UTC+7)
0 2 * * * /opt/lawu/scripts/backup_lawu.sh

# Laporan error harian pukul 07:00 WIB
0 7 * * * /opt/lawu/scripts/send_error_report.sh

# Jaga file watcher tetap hidup (restart jika mati)
@reboot /usr/bin/screen -dmS lawu-watch /opt/lawu/scripts/watch_ebooks.sh

# (OPSIONAL) Auto-update index Tantivy setiap 6 jam (jika watcher mati)
0 */6 * * * curl -s -X POST http://127.0.0.1:8081/api/reindex-all >/dev/null 2>&1
```

16 Pengujian Beban (Load Testing k6)

Install k6 (AGPL-3.0, open source). Target: p95 latency /api/search < 300ms dengan 20 VUs selama 2 menit. Error rate < 1%.

16.A-install-k6.sh SH

```sh
# 16.A — Install k6 (open source AGPL-3.0, aman)

# 1) Tambah repo k6 & install
sudo gpg -k
sudo gpg --no-default-keyring --keyring /usr/share/keyrings/k6-archive-keyring.gpg \
  --keyserver hkp://keyserver.ubuntu.com:80 --recv-keys C5AD17C747E3415A36442D57A27E0310A66EA77C
echo "deb [signed-by=/usr/share/keyrings/k6-archive-keyring.gpg] https://dl.k6.io/deb stable main" \
  | sudo tee /etc/apt/sources.list.d/k6.list
sudo apt update
sudo apt install -y k6

# 2) Verifikasi
k6 version
```

~/src/load_test.js JS

```js
// ~/src/load_test.js — target: p95 /api/search < 300ms dengan 20 Virtual Users
import http from 'k6/http';
import { check, sleep } from 'k6';

export const options = {
  vus: 20,
  duration: '2m',
  thresholds: {
    http_req_duration: ['p(95)<300'],
    http_req_failed: ['rate<0.01'],   // < 1% error rate
  },
};

export default function () {
  const res = http.post('http://127.0.0.1:8081/api/search',
    JSON.stringify({ query: 'tantivy index', file_type: 'PDF', category: 'all', limit: 60 }),
    { headers: { 'Content-Type': 'application/json' } }
  );
  check(res, {
    'status 200': (r) => r.status === 200,
    'has results': (r) => r.json('results') !== null,
  });
  sleep(0.5);   // 500ms think time antara request
}

// Jalankan: k6 run ~/src/load_test.js
// Lihat report HTML: k6 run --out json=results.json ~/src/load_test.js && k6 report --format json results.json
```

17 Reverse Proxy Nginx

Satu port 80: dashboard/API Rust di root, inference llama.cpp di /v1/. Security headers, client_max_body_size 100MB untuk upload ebook, buffering off untuk streaming chat.

17-apply-nginx-config.sh SH

```
# 17 — Tulis config Nginx (heredoc — LANGSUNG JALANKAN)
# Reverse proxy: dashboard/API Rust di root, inference llama.cpp di /v1/
sudo tee /etc/nginx/sites-available/lawu > /dev/null <<'NGINX'
server {
    listen 80;
    server_name _;   # ganti dengan domain Anda di produksi

    # Logging terpisah untuk audit
    access_log /var/log/nginx/lawu_access.log;
    error_log  /var/log/nginx/lawu_error.log;

    # Body upload max 100MB (untuk ebook PDF/EPUB besar)
    client_max_body_size 100M;

    # Dashboard + API Rust (Axum :8081)
    location / {
        proxy_pass http://127.0.0.1:8081;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_read_timeout 60s;
    }

    # Inference llama.cpp (OpenAI-compatible) untuk konsol chat
    location /v1/ {
        proxy_pass http://127.0.0.1:8080;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_read_timeout 300s;   # inference bisa lama untuk prompt panjang
        proxy_buffering off;       # streaming response (SSE untuk chat)
    }

    # Static assets (cache 1 jam)
    location ~* \.(css|js|png|jpg|jpeg|gif|svg|ico|woff2?)$ {
        proxy_pass http://127.0.0.1:8081;
        expires 1h;
        add_header Cache-Control "public, immutable";
    }

    # Security headers
    add_header X-Frame-Options "SAMEORIGIN" always;
    add_header X-Content-Type-Options "nosniff" always;
    add_header X-XSS-Protection "1; mode=block" always;
    add_header Referrer-Policy "strict-origin-when-cross-origin" always;
}
NGINX

# Enable site (symlink ke sites-enabled)
sudo ln -sf /etc/nginx/sites-available/lawu /etc/nginx/sites-enabled/lawu

# Hapus default site (bentrok dengan port 80)
sudo rm -f /etc/nginx/sites-enabled/default

# Test config (harus cetak 'syntax is ok' dan 'test is successful')
sudo nginx -t

# Reload nginx jika test ok
sudo systemctl reload nginx
sudo systemctl status nginx --no-pager
```

18 Integrasi dengan Dashboard

Hubungkan dashboard ini ke server nyata: setting endpoint LLM di tab Pengaturan, arahkan fetch ke Nginx :80, setup Telegram bot token.

integrasi-dashboard.txt TXT

```
1. Buka tab "Pengaturan" di dashboard Lawu-2.5-Fine ini.
2. Isi Endpoint LLM: http://<IP-SERVER>  (Nginx :80 akan proxy ke /v1/)
   · Model: lawu-2.5-fine
   · API Key: kosong (lokal) atau token bila Anda menambah middleware.
3. Klik Simpan — konsol chat otomatis beralih dari "Simulasi (SDK)"
   ke "llama.cpp (terhubung)".
4. Untuk data pencarian nyata: arahkan fetch ke server Rust Axum Anda
   (kontrak identik: POST /api/search, GET /api/stats, GET /api/conversations).
5. Telegram Bot: daftar bot via @BotFather, dapatkan token, isi di
   /etc/lawu/env (Langkah 15) lalu restart service lawu-2.5-fine.

# Test end-to-end dari laptop admin:
curl http://<IP-SERVER>/api/stats | jq
curl -X POST http://<IP-SERVER>/api/search \
  -H "Content-Type: application/json" \
  -d '{"query":"machine learning","limit":10}' | jq

curl http://<IP-SERVER>/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{"model":"lawu-2.5-fine","messages":[{"role":"user","content":"Halo Lawu"}],"max_tokens":50}' | jq
```

19 Final Go-Live Checklist

Audit terakhir SEBELUM buka server ke publik. Cek sistem, disk, firewall, services, ports, API health, cron, backup, memory. Semua harus sehat.

> **Jalankan checklist ini sebagai verifikasi akhir**
>
> Skrip ini akan cetak status real-time dari semua komponen. Periksa setiap baris — jika ada `✗` atau service `inactive`, FIX dulu sebelum buka port 80 ke publik.

19-go-live-checklist.sh SH

```sh
# 19 — Final Go-Live Checklist (jalankan SEBELUM buka ke publik)
# Setiap item harus ✓. Jika ada ✗, FIX dulu sebelum go-live.

echo "=== LAWU-2.5-FINE GO-LIVE CHECKLIST ==="
echo ""

echo "─── 1. SISTEM ───"
echo "1.1 Debian 13 (trixie): $(cat /etc/debian_version)"
echo "1.2 Kernel: $(uname -r)"
echo "1.3 Uptime: $(uptime -p)"
echo "1.4 Load average: $(cat /proc/loadavg | awk '{print $1, $2, $3}')"
echo ""

echo "─── 2. DISK & FILESYSTEM ───"
df -hT / /mnt/data 2>/dev/null
echo "SSD TRIM timer: $(sudo systemctl is-active fstrim.timer)"
echo ""

echo "─── 3. FIREWALL & SECURITY ───"
echo "3.1 UFW status:"
sudo ufw status | head -5
echo "3.2 fail2ban jails:"
sudo fail2ban-client status 2>/dev/null | grep -i jail || echo "   (fail2ban tidak aktif)"
echo "3.3 SSH root login disabled: $(grep -E '^PermitRootLogin' /etc/ssh/sshd_config | head -1)"
echo "3.4 SSH password auth disabled: $(grep -E '^PasswordAuthentication' /etc/ssh/sshd_config | head -1)"
echo ""

echo "─── 4. USER lawu ───"
id lawu
echo "Linger enabled: $(loginctl show-user lawu 2>/dev/null | grep Linger || echo 'no')"
echo ""

echo "─── 5. SERVICES ───"
for svc in llama-lawu lawu-2.5-fine nginx fail2ban; do
  status=$(sudo systemctl is-active "$svc" 2>/dev/null)
  enabled=$(sudo systemctl is-enabled "$svc" 2>/dev/null)
  echo "  $svc: active=$status, enabled=$enabled"
done
echo ""

echo "─── 6. PORTS LISTENING ───"
sudo ss -tlnp | grep -E ':(80|443|8080|8081)\b' | awk '{print "  "$4" "$6}'
echo ""

echo "─── 7. API HEALTH CHECK ───"
echo "7.1 Axum /api/stats:"
curl -s -o /dev/null -w "   HTTP %{http_code} in %{time_total}s\n" http://127.0.0.1:8081/api/stats
echo "7.2 Axum /api/search:"
curl -s -o /dev/null -w "   HTTP %{http_code} in %{time_total}s\n" \
  -X POST http://127.0.0.1:8081/api/search \
  -H "Content-Type: application/json" -d '{"query":"test","limit":1}'
echo "7.3 llama.cpp /v1/models:"
curl -s -o /dev/null -w "   HTTP %{http_code} in %{time_total}s\n" http://127.0.0.1:8080/v1/models
echo "7.4 Nginx /api/stats (via reverse proxy):"
curl -s -o /dev/null -w "   HTTP %{http_code} in %{time_total}s\n" http://127.0.0.1/api/stats
echo ""

echo "─── 8. CRON JOBS ───"
sudo crontab -l -u lawu 2>/dev/null | grep -v '^#' | grep -v '^$' || echo "   (tidak ada cron lawu)"
echo ""

echo "─── 9. BACKUP & LOGS ───"
echo "9.1 Backup terbaru:"
ls -lh /mnt/data/backup/lawu_*.tar.gz 2>/dev/null | tail -3 || echo "   (belum ada backup)"
echo "9.2 Log terbaru:"
ls -lh /var/log/lawu/ 2>/dev/null || echo "   (/var/log/lawu tidak ada)"
echo "9.3 Journald disk usage:"
sudo journalctl --disk-usage
echo ""

echo "─── 10. MEMORY & PERFORMANCE ───"
free -h
echo ""
echo "Top 5 proses by RAM:"
ps aux --sort=-%mem | head -6 | awk '{print "  "$2" "$3"% "$4"% "$11}'
echo ""

echo "=========================================="
echo "✅ GO-LIVE CHECKLIST SELESAI"
echo "   Periksa setiap item di atas — semua harus sehat (active, HTTP 200, ada backup)."
echo "   Jika semua ✓, server siap menerima traffic publik."
echo "=========================================="
```

✓ Setelah Langkah 19 Selesai

Server Lawu-2.5-Fine Anda siap menerima traffic publik via port 80 (Nginx). Inference & API tetap bind localhost-only (Paten keamanan). Backup harian aktif, fail2ban menjaga SSH, UFW membatasi port terbuka. **Paten 1-7 terpenuhi semua:** Rust backend, murni CPU, Linux only, open source, parallel Rayon, Islam Ahlussunnah wal Jama'ah.